Fundamental Research 5 (2025) 818–829

Contents lists available at ScienceDirect

Fundamental Research

journal homepage: http://www.keaipublishing.com/en/journals/fundamental-research/

Article

Graph contrastive learning with node-level accurate difference

Pengfei Jiao[a,c], Kaiyan Yu[a], Qing Bao[a,*], Ying Jiang[a], Xuan Guo[b], Zhidong Zhao[a]

[a] School of Cyberspace, Hangzhou Dianzi University, Hangzhou 310018, China
[b] College of Intelligence and Computing, Tianjin University, Tianjin 300350, China
[c] Data Security Governance Zhejiang Engineering Research Center, Hangzhou Dianzi University, Hangzhou 310018, China

ARTICLE INFO

Article history:
Received 4 February 2024
Received in revised form 11 June 2024
Accepted 30 June 2024
Available online 3 September 2024

Keywords:
Graph neural network
Graph contrastive learning
Accurate difference measure
Node representation learning
Pretext task design

ABSTRACT

Graph contrastive learning (GCL) has attracted extensive research interest due to its powerful ability to capture latent structural and semantic information of graphs in a self-supervised manner. Existing GCL methods commonly adopt predefined graph augmentations to generate two contrastive views. Subsequently, they design a contrastive pretext task between these views with the goal of maximizing their agreement. These methods assume the augmented graph can fully preserve the semantics of the original. However, typical data augmentation strategies in GCL, such as random edge dropping, may alter the properties of the original graph. As a result, previous GCL methods overlooked graph differences, potentially leading to difficulty distinguishing between graphs that are structurally similar but semantically different. Therefore, we argue that it is necessary to design a method that can quantify the dissimilarity between the original and augmented graphs to more accurately capture the relationships between samples. In this work, we propose a novel graph contrastive learning framework, named Accurate Difference-based Node-Level Graph Contrastive Learning (DNGCL), which helps the model distinguish similar graphs with slight differences by learning node-level differences between graphs. Specifically, we train the model to distinguish between original and augmented nodes via a node discriminator and employ cosine dissimilarity to accurately measure the difference between each node. Furthermore, we employ multiple types of data augmentation commonly used in current GCL methods on the original graph, aiming to learn the differences between nodes under different augmentation strategies and help the model learn richer local information. We conduct extensive experiments on six benchmark datasets and the results show that our DNGCL outperforms most state-of-the-art baselines, which strongly validates the effectiveness of our model.

1. Introduction

Graph neural networks (GNNs) [1–3] have become the mainstream framework for graph representation learning. GNNs have achieved great success in modeling real-world graph-structured data such as social networks [4], knowledge graphs [5], biological networks [6], and molecular graphs [7]. They can iteratively learn representations of target dimensions by transforming and aggregating neighbor information. However, previous graph representation learning methods heavily relied on abundant labeled data, which are often scarce and expensive. As a result, self-supervised learning methods [8,9] for pre-training GNNs without the necessity for labeled data have drawn significant attention.

Recently, graph contrastive learning (GCL) [10–12], one branch of the graph self-supervised learning methods, has garnered considerable research interest as it successfully applies contrastive learning from the fields of computer vision (CV) and natural language processing (NLP) to graph data. The main idea of GCL is to learn representations of contrastive views by pulling together semantically similar (positive) pairs and pushing away semantically dissimilar (negative) pairs. Most existing GCL methods follow a unified paradigm: Firstly, they employ various graph augmentation strategies, such as node dropping [13], edge perturbation [14], attribute masking [12], subgraph sampling [15], and graph diffusion [16], to generate multiple contrastive views. Subsequently, they design various contrastive pretext tasks between the contrastive views to enrich the supervisory signals. Finally, they apply widely used contrastive losses, such as Donsker–Varadhan estimator [17], Jensen–Shannon divergence [18], InfoNCE [19], and Triplet Loss [20], to maximize the agreement between the contrastive views based on the principle of mutual information maximization. Despite the vigorous development of GCL, this standard paradigm still faces challenges in designing contrastive pretext tasks.

The existing GCL methods have not precisely characterized the relationships between samples when designing the pretext task, as they overlook the differences between the original graph and the augmented

* Corresponding author.
E-mail address: qbao@hdu.edu.cn (Q. Bao).

https://doi.org/10.1016/j.fmre.2024.06.013

graph, as well as among different augmented graphs. *So why do we assert there are differences between the graphs?* Due to the invariance of image semantics under various transformations, image data augmentation has been widely used for generating contrastive views [21]. However, applying data augmentation on graphs is more challenging than on images. In the field of graph learning, dropping an edge may disrupt the key semantic relevant to downstream tasks. For example, in molecular structures, removing an edge may break a crucial chemical bond, making two molecules entirely different. Nevertheless, most existing graph data augmentation strategies (e.g., node dropping or edge perturbation) employ random perturbations of the graph topology to generate contrastive views [10–13], failing to fully preserve the semantic information of the original graph. Therefore, differences arise between the original graph and the augmented graph. Moreover, the research [13] shows that employing different augmentation strengths (e.g., edge perturbation ratio or attribute masking ratio) or combining different types of augmentation strategies for contrastive learning can affect the performance of the model. This suggests the existence of differences between augmented graphs as well. However, previous methods do not propose a solution to these differences that would alter the properties of the original graph. They often assume similarity between the graphs before and after data enhancement. Consequently, this assumption in existing GCL methods may lead to the collapse of representations for two semantically dissimilar graphs into a similar representation.

To address this problem, a recent work [22] has proposed a self-supervised learning approach aiming to endow GNNs with the ability of discriminating the graph-level differences among different graphs. However, there is still room for improvement in this work. At first, this work focuses on graph-level differences and could not capture the personalized characteristics of nodes. However, for tasks like node classification, where relationships between nodes significantly impact the overall graph structure, focusing on the local structure between nodes is crucial. This allows the model to better capture local information within the graph. Secondly, this approach learns differences between graphs by employing a single augmentation strategy to various extents. We contend that relying solely on a single augmentation strategy to capture discrepancies may lack comprehensiveness. Current GCL methods [11–13] often combine multiple augmentation strategies to generate contrastive views. Therefore, we propose combining different types of augmentation strategies to measure node-level differences between views, allowing the model to learn richer discrepancies.

In this work, we propose a novel framework for GCL to implement our idea, named as Accurate **D**ifference-based **N**ode-Level **G**raph **C**ontrastive **L**earning (DNGCL). Our approach contrasts the original and augmented views to learn the differences between them. Specifically, in order to evaluate the impact of different graph data augmentation strategies on graph properties, we initially apply multiple types of graph data augmentation to the original graph to generate different augmented views, and then learn the node-level differences between these views. To achieve this, we introduce a node discriminator learning how to distinguish between original nodes and augmented nodes. However, merely knowing that two nodes are different is insufficient, we also need to understand the precise amount of difference between them. Thus, we utilize cosine dissimilarity to compute the accurate difference for each node based on the feature matrix and adjacency matrix of the original graph and the augmented graphs. We then constrain the distance between the original and augmented nodes in the embedding space to be proportional to the magnitude of the difference, allowing the model to learn subtle differences that might have a significant impact on the properties of the graph. Additionally, capturing local information in the graph through node-level difference learning is not sufficient. We also introduce a cross-scale contrastive mechanism inspired by prior work [10,11], which compares node-level embeddings with graph-level representations to help the graph encoder learn both local and global semantic information.

The contributions of this work can be summarized as follows:

- We highlight a key limitation in most GCL methods, which assumes that augmented graphs can fully preserve the semantics of the original graph. This oversight neglects the potential compromise of original graph properties due to data augmentation. In response, we propose a new design of a contrastive pretext task between the original view and the augmented view.
- We propose a novel GCL framework named DNGCL, aiming to learn node-level differences between the original graph and its augmented counterpart. To the best of our knowledge, our work represents the first attempt to measure the relationship between nodes through differences, which makes the graph encoder better capture the local information in the graph by learning the local structure between nodes.
- We combine various strategies to generate multiple augmented graphs, aiming to evaluate the impact of different augmentation strategies on the properties of the original graph. This enables the model to capture more comprehensive discrepancies.
- We conduct extensive experiments on six public graph datasets, and the proposed DNGCL outperforms the state-of-the-art baselines, which demonstrates the effectiveness of our model from various aspects.

2. Related work

2.1. Graph representation learning

In recent years, graph representation learning (also called network embedding) [23,24], which learns representations of nodes/edges in a lower-dimensional space, has demonstrated its effectiveness for various graph mining and graph analysis tasks. Early graph representation learning methods focus on preserving the structural information of the graph. For example, DeepWalk [25] uses random walk to generate node sequences and then employs the skip-gram model to learn the cooccurrence of nodes within a window, thus capturing the local structures. LINE [26] preserves both the first- and second-order structure similarities. Node2Vec [27] builds on DeepWalk by allowing nodes to choose between Depth-First Sampling and Breadth-First Sampling during random walks, striking a balance between local and global graph structure in the embeddings.

With the development of deep learning, the emerging GNNs show powerful capability in combining the network structures and node attributes. Graph convolutional network (GCN) [28] is one of the most representative works, which extends neural networks to graphs by aggregating information from neighboring nodes, making it effective for node and graph-level tasks. GraphSAGE [29] generates node embeddings by sampling and aggregating features from local neighborhoods, offering a versatile solution for inductive graph learning. Graph attention network (GAT) [30] introduces attention mechanisms to learn the importance of neighbors for each node, excelling in tasks that require capturing fine-grained relationships. Graph isomorphism network (GIN) [31] introduces the idea of graph isomorphism and learns node representations by iteratively applying graph isomorphism operations to aggregate information from neighboring nodes.

However, most of the above methods are based on message passing neural networks (MPNNs), and research [31,32] has shown that MPNNs have limited expressive power. In recent years, a significant amount of work has focused on developing GNNs with better expressiveness. For instance, the work [33] introduced a novel spectral approach, which utilizes the eigenspaces of networks to reveal overlapping and hierarchical community structures more precisely. This approach leverages the communicability matrix and an agglomerative clustering algorithm to discover hierarchical communities, significantly improving the precision of community detection compared to traditional spectral algorithms. Additionally, subgraph GNNs are an emerging class of higher-order GNNs that compute a feature representation for each subgraph-node pair. Researchers [34] developed a principled class of subgraph GNNs,

called ESAN, which first introduced the cross-graph global aggregation into the network design, demonstrating enhanced performance in capturing complex structural information.

These methods collectively advance graph representation learning, offering efficient and powerful tools for enhancing graph data representation. Typically, we use these methods as the backbone encoders for graphs, combining them with graph self-supervised learning to better learn useful representations of graph structures in scenarios with scarce labeled data.

2.2. Graph contrastive learning

Graph contrastive learning (GCL) [35–37] has become one of the most popular self-supervised learning methods. Its main idea is to learn representations by pulling together semantically similar (positive) pairs and pushing away semantically dissimilar (negative) pairs. GCL mainly consists of three modules, e.g., data augmentation, pretext tasks, and contrastive objectives. In general, the contributions of existing work can be essentially summarized as innovations in these three modules. For example, in the case of data augmentation strategies, GRACE [12] proposes enriching node context by generating augmented graphs from both network structure and node attributes perspectives. GCA [38] proposes to keep important structures and attributes unchanged, while perturbing possibly unimportant edges and features. The pretext tasks constructed by existing GCL methods can be categorized into two types: same-scale and cross-scale contrastive learning. For example, GraphCL [13] adopts SimCLR [39] to form its contrastive pipeline which pulls the graph-level representations of two views closer. DGI [10] is proposed to maximize the mutual information between the graph-level and node-level representations of the same graph as positive view pairs. MVGRL [11] maximizes the mutual information between the cross-view representations of nodes and graphs. Lately, efforts have been made to improve the performance of contrastive learning by introducing novel contrastive learning losses. NCLA [40] proposes a new neighbor contrastive loss for node-node GCL. Furthermore, ASP [41] preserves both attribute and structure information and achieves competitive performance independent of homophily level. CSGCL [42] believes that the underlying community semantics of a graph have an impact on graph representation, they define “community strength” to measure the difference of influence among communities throughout the learning process.

In the field of GCL, several studies have proposed innovative methods to enhance the discriminability and quality of graph representations. For example, jNMF-GCL [43] combines nonnegative matrix factorization and contrastive learning to learn vertex features that preserve conserved structures in multi-layer networks. MRL_CAL [44] employs adversarial learning and contrastive learning to simultaneously learn multi-level features from incomplete multi-view clustering, addressing data restoration and representation consistency issues. UGCF [45] introduces a unified GCL framework that jointly learns data restoration, graph contrastive denoising, and clustering, significantly improving feature discriminability and clustering performance. In dynamic community detection, jNCDC [46] combines nonnegative matrix factorization and GCL to effectively capture temporal dynamics and vertex-level relationships, enhancing detection accuracy. These studies collectively advance the performance of complex network analysis tasks by integrating contrastive learning with other techniques.

The aforementioned GCL methods assume that augmented graphs could preserve the semantics of the original graph. However, typical data augmentation strategies in GCL (e.g., random edge dropping) may alter the properties of the original graph. In other words, these approaches fail to consider the differences between the original graph and augmented graphs, as well as among different augmented graphs. In this case, previous GCL methods may lead to the collapse of representations for two semantically dissimilar graphs into a similar representation. Therefore, we need to design a new GCL framework that can capture the differences between two views.

Table 1
Notations used in this paper.

Notations	Descriptions
$\mathcal{G}$	The set of graphs, $\mathcal{G} = \{g_0, g_1, \ldots, g_L\}$
g	A graph $g = (\mathcal{V}, \mathcal{E})$
L	Number of types of augmentation strategies
l	The augmented graph index, $1 \le l \le L$
g_0, g_l	The original & the lth augmented graph
$\mathcal{V}$	The set of nodes in graph g
v_i	A node $v_i \in \mathcal{V}$
$\mathcal{E}$	The set of edges in graph g
$e_{i,j}$	An edge $e_{i,j} \in \mathcal{E}$ between node v_i and node v_j
N	Number of nodes, $N = \lvert\mathcal{V}\rvert$
M	Number of edges, $M = \lvert\mathcal{E}\rvert$
$\mathbf{A} \in \{0, 1\}^{N\times N}$	Graph adjacency matrix
d	Dimension of node feature vectors
D	Dimension of node embeddings
$\boldsymbol{x}_i \in \mathbb{R}^d$	Feature vector of node v_i
$\mathbf{X} \in \mathbb{R}^{N\times d}$	Node feature matrix, $\mathbf{X} = [\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N]$
$\boldsymbol{h}_i^{(0)}, \boldsymbol{h}_i^{(l)} \in \mathbb{R}^D$	Node embedding of node v_i in original graph & augmented graph
$\boldsymbol{h}_g^{(0)}, \boldsymbol{h}_g^{(l)} \in \mathbb{R}^D$	Graph-level representation of original graph & augmented graph
$\mathbf{H} \in \mathbb{R}^{N\times D}$	Node embedding matrix, $\mathbf{H} = [\boldsymbol{h}_1, \boldsymbol{h}_2, \ldots, \boldsymbol{h}_i, \ldots, \boldsymbol{h}_N]$
$f_\theta(\cdot)$	Node-level encoder to output $\mathbf{H} = f_\theta(\mathbf{A}, \mathbf{X})$
$f_\phi(\cdot)$	The projection head
θ, ϕ	Learnable model parameters
$\mathcal{P}(\cdot)$	The graph pooling (readout) function
$\mathcal{T}(\cdot)$	Data augmentations
d_i	Cosine dissimilarity of node v_i between two views
s_i	Embedding-level distance of node v_i between two views

3. Notations and preliminaries

3.1. Notations

Let $g = (\mathcal{V}, \mathcal{E})$ denote a graph, where $\mathcal{V} = \{v_1, \ldots, v_N\}(|\mathcal{V}| = N)$ is the set of nodes and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}(|\mathcal{E}| = M)$ is the set of edges. $\mathbf{A} \in \{0, 1\}^{N\times N}$ represents the adjacency matrix of g, where $\mathbf{A}_{i,j} = 1$ if there exists an edge between node v_i and v_j, i.e., $e_{i,j} \in \mathcal{E}$, otherwise $\mathbf{A}_{i,j} = 0$. And $\mathbf{X} \in \mathbb{R}^{N\times d}$ is the node feature matrix, where d is the feature dimension. Each node v_i is associated with a feature vector $\boldsymbol{x}_i$, which is the ith row of $\mathbf{X}$. The notations used in this paper are illustrated in Table 1.

3.2. Graph neural networks

Graph neural networks [1,28,30] generate node-level embedding $\boldsymbol{h}_i$ for node v_i through aggregating the node features of its neighbors. Each layer of GNNs serves as an iteration of aggregation, such that the node embedding after the kth layers aggregates the information within its k-hop neighborhood. A general GNN framework involves two key computations for each node v_i at every layer: (1) Aggregate operation: aggregating messages from neighborhood $\mathcal{N}_i$; (2) Update operation: updating node representation from its representation in the previous layer and the aggregated messages. The kth layer of GNNs can be formulated as:

$$\boldsymbol{a}_i^{(k)} = f_A^{(k)}(\{\boldsymbol{h}_j^{(k-1)} : v_j \in \mathcal{N}_i\}) \tag{1}$$

$$\boldsymbol{h}_i^{(k)} = f_U^{(k)}(\boldsymbol{h}_i^{(k-1)}, \boldsymbol{a}_i^{(k)}) \tag{2}$$

where $f_A^{(k)}$ denotes an aggregate function that aggregates messages from the node's neighbors, $f_U^{(k)}$ denotes an update function that updates a representation of the given node along with its neighbors' representations, $\mathcal{N}_i$ denotes a set of neighbors of the node v_i, and k denotes a kth layer of GNNs.

Furthermore, for graph-level downstream tasks such as graph classification, a readout function and a projection head are required to aggregate node features to obtain a graph-level representation $\boldsymbol{h}_g$, as follow:

$$\boldsymbol{z}_g = \mathcal{P}(\{\boldsymbol{h}_i^{(k)} : v_i \in \mathcal{V}\}) \tag{3}$$

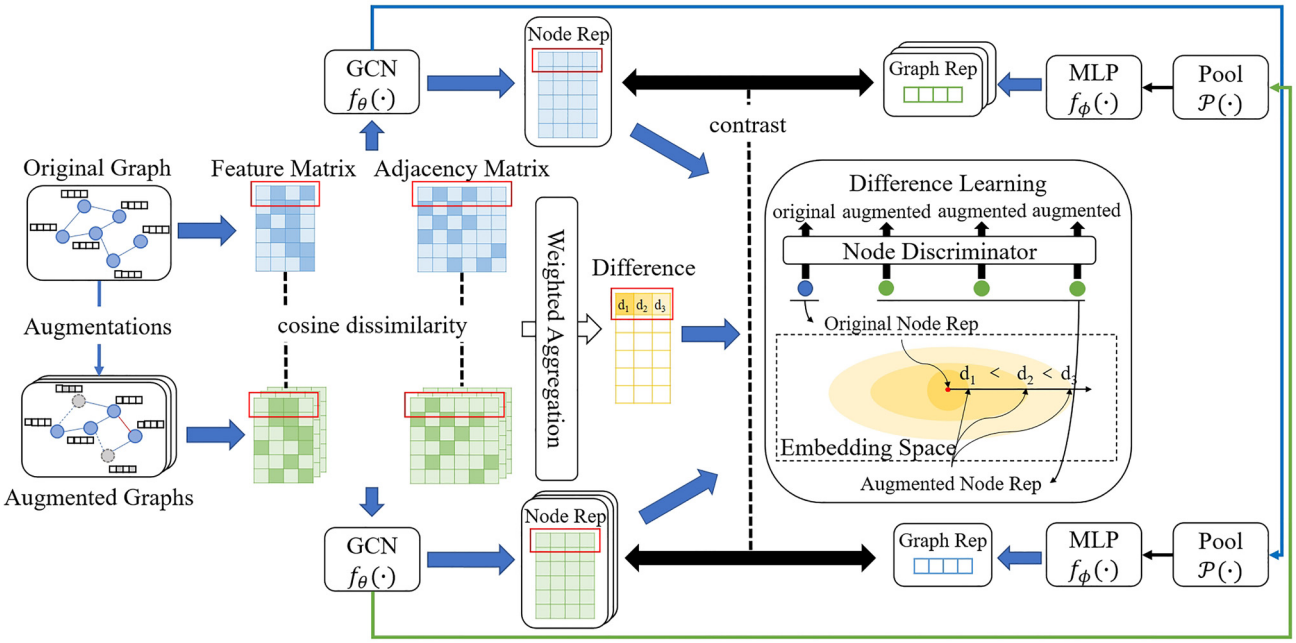

Fig. 1. The overall framework of our proposed DNGCL. We first generate several augmented views through multiple data augmentation strategies, feeding them with the original view into a shared GCN encoder to learn representations. Instead of directly contrasting these graph views, we employ an accurate difference learning module, including a node discriminator and a cosine dissimilarity-based distance constraint to characterize node relationships. Finally, we utilize a cross-scale contrastive mechanism to help the encoder learn more expressive node representations.

$$\boldsymbol{h}_g = f_\phi(\boldsymbol{z}_g) \tag{4}$$

where $\mathcal{P}$ denotes a graph pooling (readout) function, f_ϕ denotes a projection head, which is commonly implemented using a multi-layer perceptron (MLP) as its structure.

4. Method

In this section, we propose a novel graph contrastive learning framework, Accurate Difference-based Node-Level Graph Contrastive Learning (DNGCL). It helps the model distinguish similar graphs with slight differences by learning node-level differences between graphs. The overall structure of DNGCL is shown in Fig. 1, which consists of the following components:

Multiple Data Augmentation Strategies: They are utilized to generate different types of augmented graphs. We simultaneously augment both the features and structure of the graph. In this paper, we select three common augmentation strategies, i.e., $L = 3$, and for ease of illustration, we categorize these three augmentation strategies as perturbation-based, reconstruction-based, and generation-based.

Accurate Difference Learning Module: It is designed to learn node-level differences between the original graph and the augmented graph, measuring relationships between nodes through differences to mitigate the disruption of graph properties caused by data augmentation.

Cross-Scale Contrastive Mechanism: It helps graph encoders learn both local and global information about a graph by contrasting node representations from one view with graph representation from another view and vice versa. The contrastive views in this paper include the original view and the augmented view.

4.1. Augmentations

Due to the varying impact of different data augmentation strategies on the properties of the original graph, combining multiple data augmentation strategies enables the model to learn different types of differences, thereby capturing richer latent information. Therefore, we apply three advanced data augmentation strategies commonly used in current GCL works [38,47,48] to generate augmented views in two contrastive views. For ease of illustration, the data augmentation strategies used in this paper are categorized into three types as shown in Fig. 2: perturbation-based, reconstruction-based, and generation-based. We augment both the features and structure of the graph.

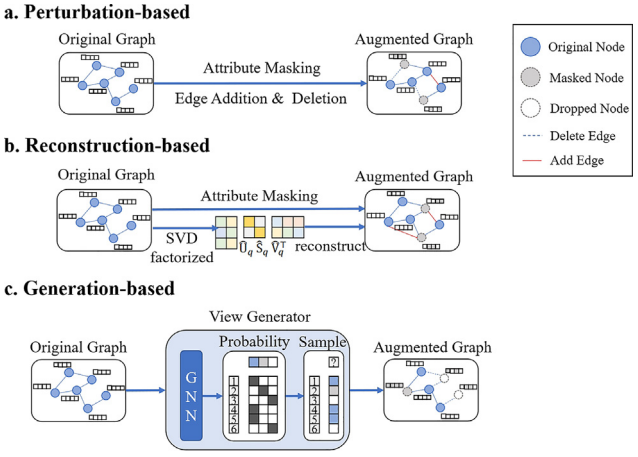

Fig. 2. The details of the three augmentation strategies. (a) For the perturbation-based strategy, we apply attribute masking to the feature matrix and add or delete edges in the adjacency matrix. (b) For the reconstruction-based strategy, we apply attribute masking to the feature matrix and perform SVD matrix decomposition to reconstruct the adjacency matrix. (c) For the generation-based strategy, we employ a learnable view generator to select an augmentation operation for each node.

4.1.1. Perturbation-based

Perturbation-based data augmentation refers to randomly masking certain feature dimensions of nodes in the given graph and partially modifying the adjacency matrix of the given graph by randomly deleting and adding a portion of edges. This strategy effectively captures local

variations in the graph, helping the model learn the randomness and uncertainty of node-level differences.

Formally, for attribute masking, we first sample a random mask vector, denoted as $\boldsymbol{m} \in \{0, 1\}^d$. Each dimension of this vector is drawn independently from a Bernoulli distribution, i.e., $m_i \sim Bern(1 - p_i)$, $\forall i$, where p_i represents the probability of each attribute being masked. This probability should reflect the importance of the ith dimension of node features. Then, given the input feature matrix $\mathbf{X}$, we define the attribute masking process as follows:

$$\mathcal{T}(\mathbf{X}) = \left[\boldsymbol{x}_1 \odot \boldsymbol{m}, \boldsymbol{x}_2 \odot \boldsymbol{m}, \ldots, \boldsymbol{x}_N \odot \boldsymbol{m}\right]^T, \tag{5}$$

where $\odot$ is the Hadamard product and $[\cdot, \cdot]$ is the concatenation operator.

For structural augmentation, we partially perturb the given graph adjacency by randomly adding or dropping a certain ratio of edges. We define this process as follows:

$$\mathcal{T}(\mathbf{A}) = (\mathbf{1} - \mathbf{M}_1) \odot \mathbf{A} + \mathbf{M}_2 \odot (\mathbf{1} - \mathbf{A}), \tag{6}$$

where $\odot$ is the Hadamard product, $\mathbf{M}_1$ and $\mathbf{M}_2$ are edge dropping and adding matrices where $\mathbf{M}_{i,j} = \mathbf{M}_{j,i} = 1$ if the edge between node v_i and v_j will be perturbed, otherwise $\mathbf{M}_{i,j} = \mathbf{M}_{j,i} = 0$. Given the perturbation ratio r, elements in $\mathbf{M}_1$ and $\mathbf{M}_2$ are set to 1 with a probability r and 0 with a probability $1 - r$.

4.1.2. Reconstruction-based

Reconstruction-based data augmentation refers to the reconstruction of the adjacency matrix of a given graph through matrix factorization. This process achieves dimensionality reduction and denoising, aiding in the identification of potential structures within the adjacency matrix. It helps the model better understand the global structure and local patterns of the graph, thereby enhancing its ability to model graph data.

For attributive augmentation, we adopt the same operations as perturbation-based augmentation. And for structural augmentation, we employ the Singular Value Decomposition (SVD) scheme [49,50] used in LightGCL [47] to reconstruct the adjacency matrix. SVD is a mathematical technique that decomposes a matrix into the product of three separate matrices: $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, where $\mathbf{U}$ and $\mathbf{V}$ are orthogonal matrices, and $\mathbf{\Sigma}$ is a diagonal matrix containing the singular values of the original matrix. The column vectors (row vectors) of the orthogonal matrices are standard orthogonal bases that indicate the principal directions in the structure of the graph. The singular values in the diagonal matrix indicate the importance of the structure of the graph. The size of the singular values determines their contribution to the original matrix $\mathbf{A}$. The larger the singular values the more important the corresponding features.

In this paper, we reconstruct an approximation of the original adjacency matrix by selecting the largest q singular values and their corresponding columns from matrices $\mathbf{U}$ and $\mathbf{V}$, i.e., $\tilde{\mathbf{A}} = \mathbf{U}_q\mathbf{\Sigma}_q\mathbf{V}_q^T$. This operation essentially preserves the most crucial structural information in the graph representation, removing some noise and less important details, while simultaneously reducing the dimensionality of the original adjacency matrix. In this way, the reconstructed matrix $\tilde{\mathbf{A}}$ is a low-rank approximation of the adjacency matrix $\mathbf{A}$, for it holds that $rank(\tilde{\mathbf{A}}) = q$.

4.1.3. Generation-based

Generation-based data augmentation refers to learning a node-level augmentation operation probability distribution through a learnable graph view generator. This enables the selection of the most suitable augmentation operation for each node, where augmentation operations include keep, mask, and drop. This strategy allows the model to capture more complex and higher-order node-level differences by learning to generate views with particular characteristics.

According to AutoGCL [48], they employ a joint training strategy to train the learnable graph view generator, the graph encoder, and the downstream task classifier in an end-to-end manner. The model learned through this strategy enables the proposed view generator to generate augmented graphs with similar semantic information but different topological properties. In this paper, we utilize their trained model to extract augmented graphs generated by the view generator.

Here, we provide a detailed introduction to the process of the learnable graph view generator. Firstly, we utilize GNNs to obtain node embeddings from node attributes. Subsequently, for each node, we use its node feature embedding to predict the probability of selecting a particular augmentation operation. The augmentation pool for each node is keep, mask, and drop. For masked node, we replace its feature with *token*. For dropped node, we replace its feature with zero vector and remove all edges connected to it. Then, we sample from these probabilities using Gumble-Softmax [51] to assign each node an augmentation operation. Formally, we utilize k GIN layers to embed the original graph, thereby generating a probability distribution $\boldsymbol{h}_i^{(k)}$ for selecting each kind of augmentation for each node. For node v_i, we have the node feature $\boldsymbol{x}_i$, the augmentation choice f_i, and the function $\mathrm{Aug}(\boldsymbol{x}, f)$ for applying the augmentation. Then the augmented feature $\boldsymbol{x}_i'$ of node v_i is obtained via:

$$\boldsymbol{a}_i^{(k)} = f_A^{(k)}(\{\boldsymbol{h}_j^{(k-1)} : v_j \in \mathcal{N}_i\}) \tag{7}$$

$$\boldsymbol{h}_i^{(k)} = f_U^{(k)}(\boldsymbol{h}_i^{(k-1)}, \boldsymbol{a}_i^{(k)}) \tag{8}$$

$$f_i = \mathrm{GumbelSoftmax}(\boldsymbol{h}_i^{(k)}) \tag{9}$$

$$\boldsymbol{x}_i' = \mathrm{Aug}(\boldsymbol{x}_i, f_i) \tag{10}$$

where $\boldsymbol{a}_i^{(k)}$ denotes the hidden state of node v_i at the kth layer, $\boldsymbol{h}_i^{(k)}$ denotes the embedding of node v_i after the kth layer, the dimension of the last layer k is set to the number of augmentation operations, and f_i is a one-hot vector sampled from the distribution via gumbel-softmax.

4.2. Difference learning

This module aims to learn node-level differences between the original graph and the augmented graph, measuring relationships between nodes through differences. Firstly, we need to identify differences between the original nodes and the augmented nodes. Secondly, we constrain the distance in the embedding space between the original nodes and augmented nodes based on the exact magnitude of the difference. Essentially, difference learning can be viewed as a regularization term designed to mitigate the disruption of graph properties caused by data augmentation.

4.2.1. Discriminating original nodes from augmented nodes

First, we adopt a shared-weight GNN encoder $f_\theta(\cdot)$ to learn the node embeddings of the original graph and different augmented graphs. The objective of our node discriminator is to distinguish the original node from augmented nodes, by predicting the original node among original and augmented nodes. Specifically, we employ a learnable linear network, called the score function, to predict the probability of each node being the original node, called the score. The node with the highest probability is predicted as the original node, and the ground truth is the index of the original node, i.e., 0. Let $\boldsymbol{h}_i^{(0)}$ and $\boldsymbol{h}_i^{(l)}$ denote the node embeddings of v_i learned by an original graph g_0 and an augmented graph g_l with $l \geq 1$ respectively. Thus, the loss of the node discriminator can be defined as follows:

$$\mathcal{L}_{ND} = -\left(\frac{\exp(S_i^{(0)})}{\exp(S_i^{(0)}) + \sum_{l \geq 1} \exp(S_i^{(l)})}\right) \text{with } S_i = f_S(\boldsymbol{h}_i) \tag{11}$$

where S_i is a score of the node v_i, obtained from a learnable score function f_S of the discriminator. By training to discriminate the original node from the augmented nodes, the model enforces a separation between the embeddings of original and augmented nodes. Therefore, following the learning from the node discriminator, our model can effectively discern the differences between original node and augmented nodes.

4.2.2. Difference learning with cosine dissimilarity

The node discriminator represents the differences between original and augmented nodes by embedding them in different locations within the embedding space, but it can only recognize that there is a difference between the original and the augmented nodes, and it cannot learn exactly how dissimilar the original and the augmented nodes are to each other because all augmented nodes are considered to be equal in the discriminator learning. Thus, in order to further understand the accurate difference between different nodes, we propose a method that utilizes cosine dissimilarity to effectively compute the differences between original node and augmented nodes. In this section, we introduce how to preserve the exact amount of differences in the learned node embedding space.

Firstly, we leverage the cosine dissimilarity to calculate the difference between the original nodes and the augmented nodes. Cosine dissimilarity is a metric commonly used to quantify the dissimilarity between two vectors in a high-dimensional space. It is particularly prevalent in applications such as natural language processing and information retrieval. Formally, the cosine dissimilarity is the complement of the cosine similarity. Given two vectors $\boldsymbol{a}$ and $\boldsymbol{b}$, the cosine dissimilarity between them can be expressed as:

$$d(\boldsymbol{a},\boldsymbol{b}) = 1 - \frac{\sum_{i=1}^{n}(a_i \cdot b_i)}{\sqrt{\sum_{i=1}^{n}(a_i)^2} \cdot \sqrt{\sum_{i=1}^{n}(b_i)^2}}, \tag{12}$$

where a_i and b_i represent the elements of vectors $\boldsymbol{a}$ and $\boldsymbol{b}$, and n is the dimensionality of the vectors. In this formula, the range of cosine dissimilarity is between 0 and 2, where a higher value indicates greater dissimilarity between the vectors.

Thus, based on the feature matrix and the adjacency matrix of the original graph and augmented graph, we can calculate the feature difference and structural difference between each augmented node and the original node by comparing the corresponding row vectors of the same node, and then we aggregate the overall difference for each node through weighted aggregation, i.e., $d_i = \mathrm{WA}(d_{attr}, d_{edge})$. Specifically, we employ a linear layer to calculate weights for feature difference and structural difference for each node. We then apply a softmax function to normalize these weights. Finally, we perform weighted aggregation to obtain the overall difference.

Since multiple data augmentation strategies are applied in this paper, each node will have several different difference values. Based on the magnitudes of these differences, we design a regularization term to learn the accurate difference between original and augmented nodes in the node embedding space. Specifically, the purpose of this regularization term is to make the model learn the embedding-level distance between original and augmented nodes is proportional to their cosine dissimilarity (i.e., if the cosine dissimilarity between two nodes is larger, then the farther they are in the embedding space, and vice versa). We first let the original node as the anchor node, then we define the cosine dissimilarity and the embedding-level distance between the anchor node $\boldsymbol{h}_i^{(0)}$ and the augmented node $\boldsymbol{h}_i^{(l)}$ as d_i and s_i, respectively. As a result, our difference loss based on the cosine dissimilarity can be expressed as:

$$\mathcal{L}_{diff} = \sum_{i,j}\left(\frac{s_i}{d_i} - \frac{s_j}{d_j}\right)^2 \text{ with } s_i = f_{dist}(\boldsymbol{h}_i^{(0)}, \boldsymbol{h}_i^{(l)}), \tag{13}$$

where f_{dist} denotes the embedding-level distance between the original and augmented nodes computed with L_2-norm.

4.3. Graph contrastive learning

The purpose of introducing the cross-scale contrastive learning mechanism in this paper is to capture the global structural information of the graph by maximizing the agreement between views at different scales. This ensures that the model can not only identify local differences but also comprehensively understand the overall structure of the graph. This mechanism complements the accurate difference learning module, and together, they enhance the performance of the model. We utilize the deep InfoMax [52] approach and maximize the mutual information between node representations from one view and graph representation from another view and vice versa. Specifically, after obtaining the node embedding matrices $\mathbf{H}^{(0)}$ for the original graph and $\mathbf{H}^{(l)}$ for the augmented graphs through a shared-weight GNN encoder $f_\theta(\cdot)$, a graph pooling (readout) function $\mathcal{P}(\cdot)$ and a projection head $f_\phi(\cdot)$ are use to obtain graph-level representations $\boldsymbol{h}_g^{(0)} = f_\phi(\mathcal{P}(\mathbf{H}^{(0)}))$ and $\boldsymbol{h}_g^{(l)} = f_\phi(\mathcal{P}(\mathbf{H}^{(l)}))$. The learning objective is defined as follows:

$$\mathcal{L}_{cl} = \max_{\theta,\phi}\frac{1}{N}\sum_{v_i\in\mathcal{V}}\left[\sum_{l=1}^{L}\mathrm{MI}\left(h_i^{(0)}, h_g^{(l)}\right) + \sum_{l=1}^{L}\mathrm{MI}\left(h_i^{(l)}, h_g^{(0)}\right)\right], \tag{14}$$

where θ, ϕ are graph encoder and projection head parameters, N is the number of nodes in graph g, L is the number of types of augmentation strategies, MI represents mutual information, and $\boldsymbol{h}_i^{(0)}$ is the representations of node v_i encoded from the original view (0). To generate negative samples, we randomly shuffle the features, following the approach described in reference [10,11], which involves row-wise shuffling of the feature matrix $\mathbf{X}$. Therefore, the overall learning objective of our proposed DNGCL is given as follows:

$$\mathcal{L} = \lambda_1\mathcal{L}_{ND} + \lambda_2\mathcal{L}_{diff} + \lambda_3\mathcal{L}_{cl}, \tag{15}$$

where hyperparameters λ_1, λ_2 and λ_3 are balance parameters to each loss. We describe the entire algorithm of DNGCL in Algorithm 1.

4.4. Complexity analysis

The complexity of DNGCL is mainly composed of three parts: data augmentation, difference calculation, and the encoding process. For the data augmentation part, we employ three types of data augmentation methods. The complexity for perturbation-based augmentation includes feature masking with complexity $\mathcal{O}(dN)$ and edge perturbation with complexity $\mathcal{O}(N^2)$, resulting in a total complexity of $\mathcal{O}(N^2)$. Reconstruction-based augmentation includes feature masking with complexity $\mathcal{O}(dN)$ and adjacency matrix SVD decomposition reconstruction with complexity $\mathcal{O}(qN^2)$, resulting in a total complexity of $\mathcal{O}(qN^2)$. Generation-based augmentation uses a GIN network to learn an appropriate augmentation method for each node, with a complexity of $\mathcal{O}(M+N)$. For difference calculation, the complexity for calculating the cosine dissimilarity of the feature matrix is $\mathcal{O}(dN)$, and for the adjacency matrix, it is $\mathcal{O}(N^2)$. For the encoding process, we use a GCN encoder that follows the message-passing mechanism, with a complexity of $\mathcal{O}(M+N)$. Here, M represents the number of edges, N represents the number of nodes, d represents the feature dimension, and q represents the number of largest singular values. Overall, the complexity of DNGCL is comparable to that of the SOTA GCL methods, demonstrating a certain level of scalability.

5. Experiments

In this section, we conduct extensive experiments to validate the proposed DNGCL. We start by introducing the experimental setup. Then we show the performance comparison between DNGCL and SOTA GCL methods on node classification and node clustering tasks. We also perform ablation study and parameter sensitivity analysis to experimentally validate our motivation.

5.1. Experimental setup

5.1.1. Datasets

We evaluate our approach on six benchmark datasets. The statistics of the datasets are summarized in Table 2.

Cora, Citeseer and **Pubmed** are three widely-used citation network datasets. Nodes represent documents and edges represent citation links. Each node has a predefined feature. For these three datasets, we follow

Algorithm 1
DNGCL algorithm.

Input: Graph $g_0 = (\mathcal{V}, \mathcal{E}, \mathbf{X})$
Output: Node embedding matrix $\mathbf{H}_0$ of original graph g_0
for *epoch* $\leftarrow$ 1, 2, $\cdots$ do
  Generate multiple graph views $g_1, g_2, \cdots, g_L$ by performing data augmentations $\mathcal{T}(\cdot)$ on g_0;
  Obtain node embedding matrix $\mathbf{H}_0$ of original graph g_0 using the encoder $f_\theta(\cdot)$;
  Obtain node embedding matrix $\mathbf{H}_l$ of augmented graph g_l using the encoder $f_\theta(\cdot)$, $1 \le l \le L$;
  Compute the node discriminator objective $\mathcal{L}_{ND}$ with Eq. (11);
  Compute the cosine dissimilarity d_i of node v_i between two views with Eq. (12);
  Compute the difference learning objective $\mathcal{L}_{diff}$ with Eq. (13);
  Compute the contrastive objective $\mathcal{L}_{cl}$ with Eq. (14);
  Update parameters by applying stochastic gradient descent to minimize $\mathcal{L}$ with Eq. (15);
end for
retrun The optimal encoder parameters θ.

Table 2
Statistics of datasets used in experiments.

Dataset	#Nodes	#Edges	#Features	#Classes
Cora	2708	10,556	1433	7
Citeseer	3327	9104	3703	6
Pubmed	19,717	88,648	500	3
Photo	7650	238,162	745	8
Computers	13,752	491,722	767	10
CS	18,333	163,788	6805	15

the evaluation protocols used in GRACE [12], we choose 10% of the nodes as the training set, 10% of the nodes as the validation set, and the rest as the test set.

Photo and **Computers** [53] are two networks of co-purchase relationships constructed from Amazon. Nodes represent goods and edges indicate that two goods are frequently bought together, node features are bag-of-words encoded product reviews, and class labels are given by the product category. For these two datasets, due to the absence of commonly used dataset splits and to evaluate the robustness of the model under different dataset split ratios, we conduct experiments using three different random splits, where {40%/30%/30%, 60%/20%/20%, 80%/10%/10%} of the nodes are selected as the training, validation, and test set, respectively.

CS [54] is a co-authorship graph based on the Microsoft Academic Graph from the KDD Cup 2016 challenge. Here, nodes are authors, that are connected by an edge if they co-authored a paper, node features represent paper keywords for each author's papers, and class labels indicate the most active fields of study for each author. For this dataset, due to the absence of commonly used dataset splits and to evaluate the robustness of the model under different dataset split ratios, we conduct experiments using three different random splits, where {40%/30%/30%, 60%/20%/20%, 80%/10%/10%} of the nodes are selected as the training, validation, and test set, respectively.

5.1.2. Baselines

To evaluate the performance of DNGCL, we compared it with several state-of-the-art baselines, which can be categorized into two groups: two semi-supervised baselines including GCN and GAT; six self-supervised baselines including DGI, MVGRL, GraphCL, ASP, CSGCL, and NCLA. Details of these methods are as follows:

GCN [28] extends neural networks to graphs by aggregating information from neighboring nodes, making it effective for node and graph-level tasks.

GAT [30] introduces attention mechanisms to learn the importance of neighbors for each node, excelling in tasks that require capturing fine-grained relationships.

DGI [10] introduces a self-supervised framework for learning node embeddings by maximizing mutual information between a node and its graph neighborhood representations.

MVGRL [11] maximizes the mutual information between the node representations of one view and the graph representations of another view.

GraphCL [13] applies four different data augmentation strategies, to obtain graph representations through a self-supervised pretraining approach.

ASP [41] preserves both attribute and structure information and learns effective node representations for graphs with different levels of homophily.

CSGCL [42] captures and preserves "community strength" information throughout the learning process, enabling the model to learn more discriminative representations.

NCLA [40] generates thoroughly learnable graph augmentation by the multi-head graph attention mechanism and proposes a new neighbor contrastive loss for node-node GCL.

5.1.3. Evaluation protocol

Firstly, we learn node representations in an unsupervised manner. Subsequently, for node classification, we employ these representations to train and test an L_2-regularized logistic regression (LR) classifier. For node clustering, we evaluate the proposed method under the clustering evaluation protocol and cluster the learned representations using the K-Means algorithm. The number of clusters K is set as the number of target node classes. We use accuracy as the evaluation metric for node classification, normalized mutual information (NMI) and adjusted rand index (ARI) as the evaluation metrics for node clustering. To keep the results stable, each experiment is repeated 10 times to report the average performance.

5.1.4. Implementation details

We implement our proposed framework and some baselines using Pytorch [55] and Pytorch Geometric [56]. Our hardware configuration comprises a GeForce GTX 3090 Graphics Card with 64GB of system memory. We use the Adam optimizer to optimize the model, and we utilize a 2-layer GCN to obtain node representations. We set a patience of 20 and a maximum of 5000 epochs for early stopping. The hyperparameters of DNGCL on the six datasets are specified in Table 3, where lr represents the learning rate for model training, h represents the hidden layer dimension, D represents the embedding dimension, and λ_1, λ_2 and λ_3 respectively represent the three balance parameters of the loss function.

5.2. Performance comparision

5.2.1. Node classification

We report the performance of node classification in Table 4.We bold the best method in each row and "–" indicates that experimental results could not be obtained due to CUDA running out of memory. In addition, we record the average ranking of classification results of each method on six datasets.

Table 3
Hyperparameter settings of DNGCL.

Dataset	lr	h	D	λ_1	λ_2	λ_3
Cora	0.001	512	256	5	1e−09	40
Citeseer	0.001	512	256	5	1e−09	40
Pubmed	0.0001	256	256	5	1e−09	50
Photo	0.0001	512	256	5	1e−07	40
Computers	0.0001	256	256	5	1e−10	20
CS	0.0001	512	256	1	1e−10	50

We can find that DNGCL outperforms most state-of-the-art baselines across the six datasets, which verifies the superiority of our proposed framework. Explanations for the performance improvement can be demonstrated as follows. First, the results show that DNGCL outperforms all semi-supervised methods that adopt the label information in the learning process. In particular, on the Cora dataset, DNGCL achieves a 5.3% improvement in classification accuracy compared to GCN. Recall that DNGCL utilizes GCN as the backbone encoder to generate initial node representations. The significant improvement of DNGCL over GCN reflects that more fine-grained representations can be learned with the help of GCL. Compared to self-supervised methods, DNGCL achieves the best results on five out of the six datasets. Only on Photo, DNGCL is slightly weaker than NCLA and MVGRL at low training set ratios. Considering that NCLA generates learnable augmented views with adaptive topology by multi-head GAT, where the parameters of each view are not shared, this method suffers from a high parameter cost. Furthermore, MVGRL involves diffusion matrix operations like PPR or heat kernel, which is challenging for large graphs. By contrast, every component is shared across the graphs in DNGCL, so only a single GNN encoder is needed. Overall, the stable performance of our method against different training ratios indicates the robustness of our learned node embeddings. Then, different from DGI and MVGRL, which choose cross-scale contrastive objective to serve for encoder training, DNGCL, on this basis, also considers the impact of node-level differences between views. This is beneficial for enriching latent node representations and further enhancing performance.

5.2.2. Node clustering

In the node clustering task, for the Photo, Computers, and CS datasets, we report the results when the ratio of training set, validation set, and test set is 8:1:1. We report the performance of node clustering in Table 5. We bold the best method in each column and "–" indicates that experimental results could not be obtained due to CUDA running out of memory. In addition, we record the average ranking of clustering results of each method on six datasets.

From Table 5, we can see that DNGCL has the highest average ranking across the six datasets, indicating that our method exhibits strong generalization ability. We observe that self-supervised methods struggle to surpass semi-supervised methods in node clustering tasks. This could be attributed to the fact that semi-supervised methods leverage label information to guide representation learning, thereby improving clustering performance. However, our DNGCL, through node-level difference learning, can obtain more discriminative representations, surpassing semi-supervised methods. For instance, compared to GAT, DNGCL improves the NMI and ARI scores by up to 8.1% and 27.7% on Cora. Additionally, we find that the performance of semi-supervised methods varies widely across datasets. For example, although GAT achieves optimal performance on the Photo and Computers datasets, it performs poorly on the Citeseer dataset, with NMI and ARI scores 14.5% and 26% lower than our DNGCL, respectively. Compared to self-supervised methods, our DNGCL outperforms most of the baselines. On the Cora dataset, for example, our method achieves a 0.5% improvement over the second-best comparison on NMI and a 2.6% improvement on ARI. We believe the reason is that our proposed DNGCL, through difference learning, measures the distance relationship between original nodes and augmented nodes in the embedding space, enabling the model to learn more fine-grained node representations.

We find that although DNGCL outperforms most baseline methods in node clustering tasks, the overall performance improvement is not very pronounced. This may be because the effectiveness of node clustering tasks largely depends on the characteristics of the dataset. Some datasets may already possess well-defined structural features, allowing existing baseline methods to achieve good results, thereby limiting the improvement space for new methods. Secondly, although our method can obtain high-quality node embeddings during the pre-training, these embeddings are primarily optimized for the GCL task and may not be fully suitable for subsequent clustering tasks. GCL focuses on capturing relationships between nodes and the overall structure of the graph, while node clustering focuses on the grouping characteristics of nodes. This difference may lead to less significant improvements in clustering performance. Node clustering tasks might require more refined embedding adjustments. Therefore, we have considered some potential improvement directions, such as introducing a fine-tuning stage after obtaining initial node embeddings during pre-training. This stage would specifically optimize the node embeddings for the clustering task to enhance clustering performance.

5.2.3. Visualization

To provide a more intuitive evaluation, we conduct embedding visualization on Cora dataset. We plot learned embeddings of DGI, MVGRL, GraphCL, ASP, NCLA and DNGCL using t-SNE [57], and the results are shown in Fig. 3, in which different colors mean different labels. We can see that DGI, GraphCL and ASP are unable to distinguish nodes of the yellow type. MVGRL gives a better visualization result but the boundaries of its clusters are fairly blurry. For NCLA, the clusters are more clearly separated, but its intra-class compactness is not enough. Overall, the visualization result of DNGCL is significantly better than the baselines. Specifically, the number of the clusters is consistent with the number of the ground truth labels, the clusters are compact, and

Table 4
Accuracy (with standard deviation) of node classification (in %). The best results are highlighted in bold.

Datasets	Train	GCN	GAT	DGI	MVGRL	GraphCL	ASP	CSGCL	NCLA	DNGCL
Cora	10(%)	80.2 ± 0.7	81.8 ± 0.9	82.4 ± 0.2	81.8 ± 0.1	81.3 ± 0.2	80.9 ± 0.9	81.9 ± 0.8	82.5 ± 1.4	**85.5 ± 0.7**
Citeseer	10(%)	68.0 ± 0.5	69.0 ± 0.5	72.3 ± 0.2	72.2 ± 0.1	69.8 ± 0.3	70.3 ± 0.7	66.3 ± 0.7	70.9 ± 0.8	**77.4 ± 0.2**
Pubmed	10(%)	78.5 ± 0.5	77.9 ± 0.3	77.9 ± 0.3	80.5 ± 0.2	76.5 ± 0.4	78.8 ± 0.2	84.5 ± 0.5	79.5 ± 1.3	**84.7 ± 0.4**
Photo	40(%)	86.8 ± 3.8	93.3 ± 0.6	48.6 ± 0.1	93.1 ± 0.1	42.6 ± 0.1	83.1 ± 0.8	86.6 ± 0.7	**94.1 ± 0.4**	93.8 ± 0.2
	60(%)	77.6 ± 17.3	93.7 ± 0.5	51.4 ± 0.1	**94.4 ± 0.1**	42.3 ± 0.1	82.8 ± 1.6	90.2 ± 0.7	94.3 ± 0.3	93.9 ± 0.1
	80(%)	89.1 ± 6.5	93.8 ± 0.8	49.8 ± 0.1	94.2 ± 0.1	41.2 ± 0.1	81.8 ± 0.9	90.4 ± 0.7	94.3 ± 0.9	**94.9 ± 0.3**
Computers	40(%)	78.5 ± 6.8	88.8 ± 0.7	43.9 ± 0.1	86.6 ± 0.1	39.5 ± 0.1	71.4 ± 0.1	89.8 ± 0.4	89.4 ± 0.5	**90.0 ± 0.1**
	60(%)	69.3 ± 13.1	89.2 ± 0.4	45.4 ± 0.1	86.8 ± 0.3	39.4 ± 0.1	71.5 ± 0.2	90.2 ± 0.5	89.8 ± 0.3	**90.7 ± 0.2**
	80(%)	74.2 ± 12.4	89.1 ± 1.2	45.7 ± 0.1	86.8 ± 0.4	40.0 ± 0.1	73.3 ± 0.2	90.4 ± 0.5	89.8 ± 0.5	**90.5 ± 0.2**
CS	40(%)	93.3 ± 0.4	93.2 ± 0.4	87.9 ± 0.1	–	79.8 ± 0.2	82.9 ± 0.1	92.3 ± 0.4	93.0 ± 0.2	**93.5 ± 0.1**
	60(%)	93.9 ± 0.5	93.6 ± 0.4	87.0 ± 0.1		79.6 ± 0.3	83.6 ± 0.1	92.3 ± 0.4	93.1 ± 0.3	**94.2 ± 0.1**
	80(%)	93.6 ± 0.3	93.7 ± 0.3	86.6 ± 0.1		78.8 ± 0.3	84.4 ± 0.1	92.3 ± 0.5	93.3 ± 0.4	**95.1 ± 0.1**
Avg. Rank		5.6	4.2	6.6	5.2	8.3	6.6	4.3	3.0	**1.3**

Table 5
Node clustering performance measured by NMI and ARI (in %). The best results are highlighted in bold.

Baselines	Cora		Citeseer		Pubmed		Photo		Computers		CS		Avg. Rank	
	NMI	ARI	NMI	ARI	NMI	ARI	NMI	ARI	NMI	ARI	NMI	ARI	NMI	ARI
GCN	59.7	58.7	38.1	38.4	35.9	39.7	66.6	51.9	50.1	30.5	77.4	72.2	3.7	3.3
GAT	52.1	31.8	27.6	16.2	36.0	37.6	**76.4**	**69.0**	**62.5**	**49.5**	68.8	49.3	4.5	4.7
DGI	56.8	51.9	**44.4**	**45.2**	15.9	13.7	38.2	24.8	34.7	24.7	76.0	61.4	5.7	5.5
MVGRL	58.1	52.8	41.6	35.3	5.8	4.8	43.6	24.9	24.8	15.3		–	7.0	7.5
GraphCL	57.3	52.7	44.4	44.9	5.2	2.1	39.3	27.2	34.0	23.8	74.8	65.7	6.2	5.3
ASP	55.7	46.5	43.2	43.1	16.8	10.2	60.3	46.6	49.2	31.9	77.4	63.2	4.8	5.0
CSGCL	26.1	9.1	16.7	10.2	18.4	18.1	41.3	26.2	43.7	21.4	54.0	34.0	7.3	7.7
NCLA	59.7	56.9	42.2	38.3	32.9	29.8	70.3	60.7	54.6	38.1	73.6	55.3	3.8	4.0
DNGCL	**60.2**	**59.5**	42.1	42.2	**37.7**	**39.8**	71.4	52.4	57.5	39.1	**85.5**	**79.2**	**2.0**	**2.0**

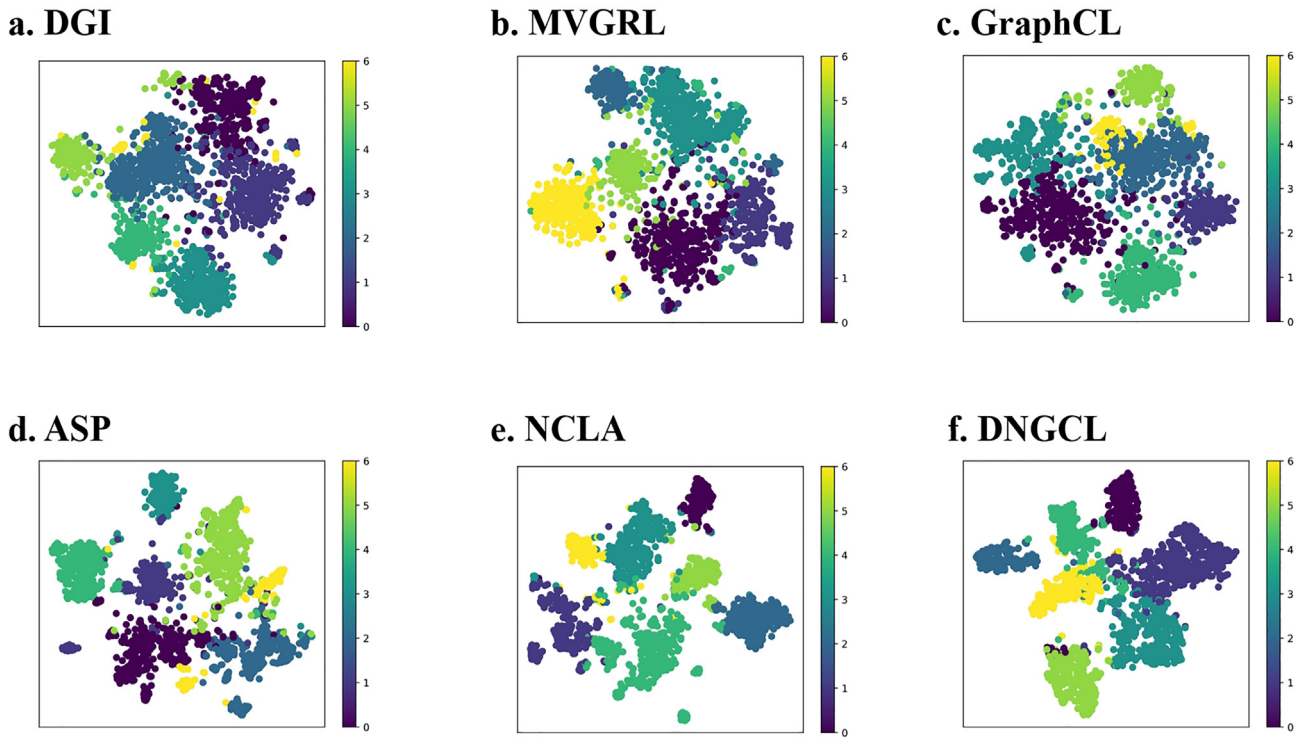

Fig. 3. Visualization of the node representations on Cora. The different colors mean different labels. Our DNGCL shows the intra-class compactness and the clear inter-class boundaries, which demonstrates its effectiveness.

the separations between the clusters are clear. This visualization result confirms that DNGCL better captures the relationships between nodes compared to the baselines, further demonstrating the effectiveness of DNGCL.

5.3. Ablation study

5.3.1. Effectiveness of the components

In this section, we conduct ablation studies to validate the contributions of each component proposed in DNGCL. Specifically, we conduct node classification experiments on Cora and Citeseer datasets by masking different components under the same hyperparameters and training scheme, where "-w/o $\mathcal{L}_{ND}$" denotes the removal of the node discriminator, "-w/o $\mathcal{L}_{diff}$" denotes the removal of the cosine dissimilarity-based difference learning, "-w/o $\mathcal{L}_{cl}$" denotes the removal of the contrastive learning module. From the results shown in Table 6a, we can see that removing any component results in a performance decline. We observe that removing the node discriminator leads to a decrease in classification accuracy by 1.3%, indicating that discriminating original nodes from augmented nodes improves performance. However, leveraging the node discriminator alone cannot learn sufficiently excellent representations. This is because, without difference learning based on cosine dissimilarity, the model fails to capture the accurate differences between augmented nodes, resulting in a decline in performance. Furthermore, we find that removing the contrastive learning module causes

Table 6
Ablation study in node classification on Cora and Citeseer.

Ablation	Cora	Citeseer
(a) Ablation study on components.		
-w/o $\mathcal{L}_{ND}$	84.2 ± 0.7	74.8 ± 1.4
-w/o $\mathcal{L}_{diff}$	84.3 ± 1.3	74.9 ± 1.0
-w/o $\mathcal{L}_{cl}$	84.9 ± 0.7	77.3 ± 1.3
DNGCL	**85.5 ± 0.7**	**77.4 ± 0.2**
(b) Ablation study on augmentation strategies.		
DNGCL-ptb	84.3 ± 0.8	75.7 ± 1.8
DNGCL-rec	77.1 ± 1.3	77.2 ± 0.9
DNGCL-gen	85.3 ± 0.5	72.1 ± 1.2
DNGCL	**85.5 ± 0.7**	**77.4 ± 0.2**

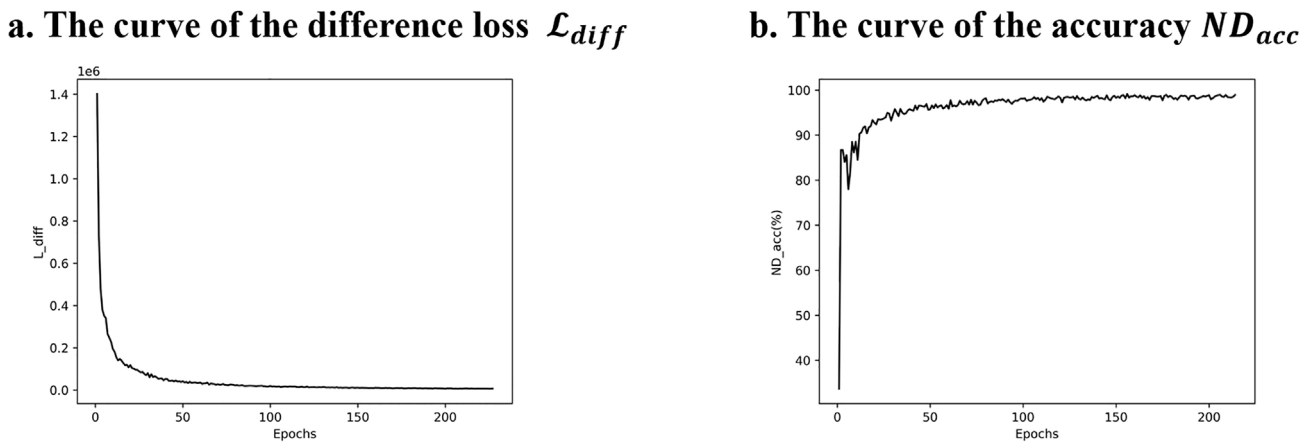

Fig. 4. Analysis on the accurate difference learning module.

a decrease in classification accuracy, but not as much as the first two, which verifies that accurate difference learning with cosine dissimilarity is important for graph modeling.

Additionally, to verify whether the accurate difference learning module successfully captures the accurate differences between views, we explore the variation of the difference loss $\mathcal{L}_{diff}$ with pre-training epochs and the accuracy of the node discriminator ND_{acc} in correctly distinguishing between original and augmented nodes during the pre-training process. According to the Eq. 13 for our difference loss L_{diff}, d refers to the amount of difference between the augmented node and the original node, which is calculated and not learnable. s refers to the distance between the augmented node and the original node in the embedding space. Since node representations are learned through GCN, this distance is learnable. Therefore, by optimizing this loss function, our goal is to ensure that the representation of an augmented node is farther from the representation of the original node when the difference between them is greater. As shown in Fig. 4a, the difference loss L_{diff} continuously decreases and eventually converges as the training epochs increase. This indicates that our accurate difference learning module successfully captures node-level accurate differences between views during training, and this process is stable and gradually optimized. Moreover, during the pre-training process, we calculated the accuracy of the node discriminator in the accurate difference learning module in correctly distinguishing between original and augmented nodes. As shown in Fig. 4b, the discriminator's accuracy gradually increases and eventually stabilizes as the training epochs increase. This indicates that as the model is trained, the performance of the node discriminator in distinguishing original nodes from augmented nodes significantly improves, thereby validating the effectiveness of the accurate difference learning module.

5.3.2. *Effectiveness of combining different data augmentation strategies*

In addition, in order to demonstrate that combining different data augmentation strategies can help the model learn richer information, we compare it with the approach of applying only a single data augmentation strategy to different extents. "DNGCL-ptb" refers to using the perturbation-based augmentation strategy for augmentation at three different extents. "DNGCL-rec" refers to using the reconstruction-based augmentation strategy for augmentation at three different extents. "DNGCL-gen" refers to using the generation-based augmentation strategy for augmentation at three different extents. Results are shown in Table 6b. We observe that combining three data augmentations performs better than adopting a single data augmentation at three different extents. We find that, on the Cora dataset, there is a significant performance decline when using only reconstruction-based data augmentation. We believe this might be because, for the Cora dataset, reconstruction-based data augmentation tends to cause greater damage to the properties of the original graph. Therefore, relying solely on this data augmentation method makes it challenging for difference learning to capture relationships between nodes. Similarly, adopting only generation-based data augmentation on the Citeseer dataset also leads to a large drop in classification accuracy. This reflects that, for different datasets, the extent of disruption caused by different data augmentation strategies to the original graph varies. It also indicates that it is necessary to combine different data augmentation strategies for difference learning.

5.4. *Parameter sensitivity*

In this section, we perform sensitivity analysis on several critical hyperparameters in DNGCL, including the hidden layer dimension h, the embedding dimension D, balance parameter λ_1, balance parameter λ_2 and balance parameter λ_3. We only change these five parameters in the sensitivity analysis, and other parameters remain the same as previously described. Experimental results are reported on the node classification task on the Cora and Citeseer datasets in Fig. 5.

The results in Fig. 5a indicate that the accuracy of node classification is relatively stable when the hidden layer dimension h changes in a certain range. The optimal performance is achieved when the dimension is set to 512. However, the performance decreases when the dimension is larger than 512. Such phenomenon suggests that an appropriate number of parameters could enhance the model capacity, but too many parameters would lead to overfitting and decrease the model generalization ability. Moreover, as shown in the Fig. 5b, the experimental results indicate: At lower embedding dimensions, the performance of the model is relatively stable but slightly lower than at higher embedding dimensions. At an embedding dimension of 256, the model achieves optimal performance, indicating that this dimension can effectively capture the structural features of the graph. When the embedding dimension is increased to 512, the performance of the model declines. This could be due to the higher embedding dimension introducing a significant amount of redundant information, increasing noise, and making it difficult for the model to extract useful features, thereby reducing the expressive power of the embedding vectors. Overall, our method exhibits robustness to the embedding dimension settings, maintaining high performance across a wide range of dimensions.

Secondly, we test the impact of three loss balance parameters on performance, and they show an overall trend of first increasing and then decreasing. As shown in Fig. 5c and e, for the Cora dataset, λ_1 and λ_3 exhibit relatively small variations, suggesting that DNGCL is not highly sensitive to these two parameters on the Cora dataset. For the Citeseer dataset, inappropriate values of λ_1 will impact the effectiveness of the

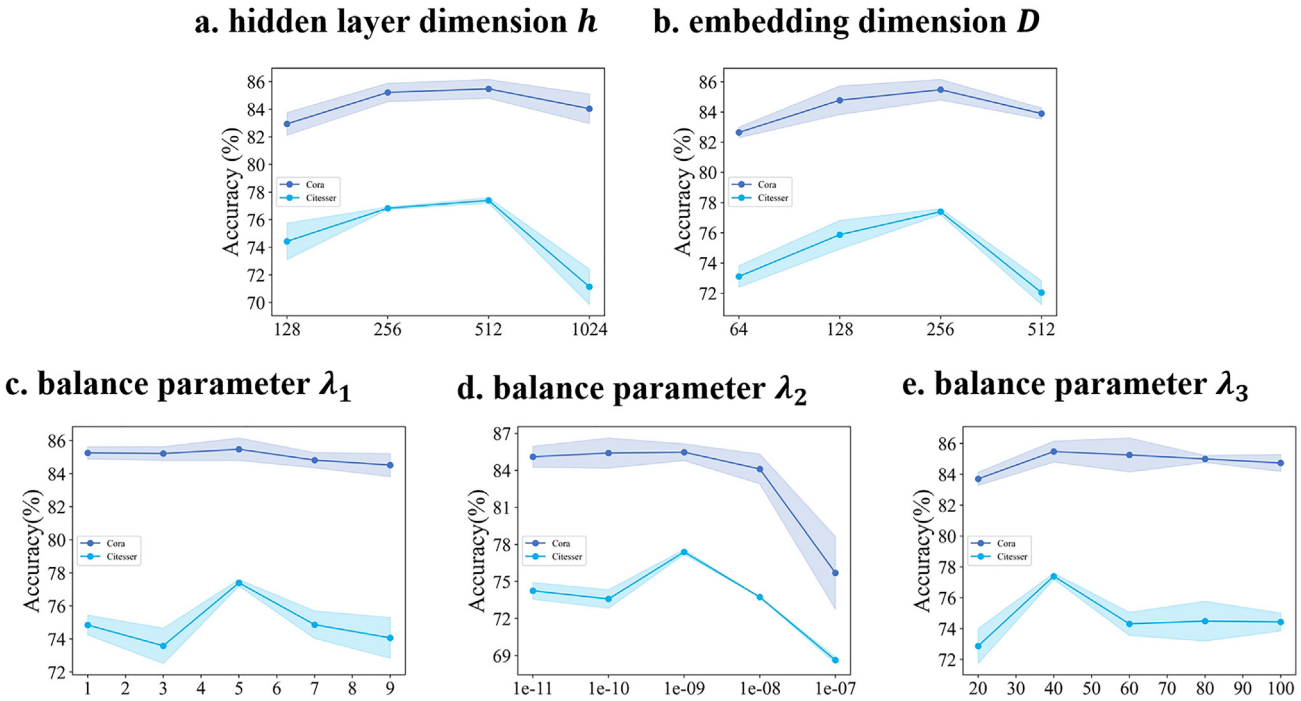

Fig. 5. Analysis on critical hyperparameters.

node discriminator, thereby reducing the performance and DNGCL can achieve satisfactory performance when λ_1 is set to 5. When λ_3 is too small, a decline in performance is observed, which may be caused by the failure of the model to effectively capture the global information of the graph. From Fig. 5d, we can find that the variation pattern of λ_2 is data-specific and an inappropriate λ_2 will deteriorate the performance. On the Cora dataset, a noticeable decline in performance occurs when λ_2 exceeds 1e-08, while on the Citeseer dataset, a significant drop is observed when λ_2 exceeds 1e-09. This is due to the fact that too large λ_2 will cause the model to overly focus on local information between nodes, neglecting the global information present in the graph. Thus we have to carefully seek a balance for λ_2 value on different datasets.

6. Conclusion

In this paper, we propose a model named DNGCL, which captures the node-level differences between the original and augmented graphs through difference learning. DNGCL aims to measure the relationship between samples by the magnitude of differences, thus enabling the model to distinguish between similar graphs with slight differences and improving its representation ability. Specifically, we use a node discriminator to distinguish the original node from augmented nodes and utilize cosine dissimilarity to quantify the magnitude of differences. Moreover, DNGCL combines multiple data augmentation strategies to generate augmented graphs. Compared to using a single data augmentation strategy, our approach can capture richer latent relationships. Our proposed DNGCL shows strong competitiveness in node classification, node clustering, and visualization tasks compared to state-of-the-art methods. Experimental results verify the effectiveness and superiority of our node-level difference learning.

There are some potential improvements to the proposed model that could be addressed in the future. For example, in addition to simply using cosine dissimilarity to calculate differences, exploring effective methods based on pattern matching can be considered for studying topological structural differences between views. Also, extending the proposed model for application in complex graphs is another avenue worth exploring.

Declaration of competing interest

The authors declare that they have no conflicts of interest in this work.

Acknowledgments

This work was supported in part by the Zhejiang Provincial Natural Science Foundation of China (LDT23F01012F01 and LDT23F01015F01), in part by the Fundamental Research Funds for the Provincial Universities of Zhejiang Grant GK229909299001-008 and the National Natural Science Foundation of China (62372146 and 61806061).

References

[1] J. Zhou, G. Cui, S. Hu, et al., Graph neural networks: A review of methods and applications, AI Open 1 (2020) 57–81.
[2] Z. Zhang, P. Cui, W. Zhu, Deep learning on graphs: A survey, IEEE Trans. Knowl. Data Eng. 34 (1) (2020) 249–270.
[3] Y. Wang, D.D. Zeng, Q. Zhang, et al., Adaptively temporal graph convolution model for epidemic prediction of multiple age groups, Fundam. Res. 2 (2) (2022) 311–320.
[4] W. Fan, Y. Ma, Q. Li, et al., Graph neural networks for social recommendation, in: The World Wide Web Conference, WWW 2019, San Francisco, CA, USA, May 13–17, 2019, 2019, pp. 417–426.
[5] J. Baek, D.B. Lee, S.J. Hwang, Learning to extrapolate knowledge: Transductive few-shot out-of-graph link prediction, in: Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6—12, 2020, virtual, 2020, pp. 546–560.
[6] G. Muzio, L. O'Bray, K. Borgwardt, Biological network analysis with deep learning, Briefings Bioinform. 22 (2) (2021) 1515–1530.
[7] Y. Xie, C. Shi, H. Zhou, et al., Mars: Markov molecular sampling for multi-objective drug discovery, in: 9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3–7, 2021, 2021, pp. 1–19.
[8] Y. Liu, M. Jin, S. Pan, et al., Graph self-supervised learning: A survey, IEEE Trans. Knowl. Data Eng. 35 (6) (2022) 5879–5900.
[9] L. Wu, H. Lin, C. Tan, et al., Self-supervised learning on graphs: Contrastive, generative, or predictive, IEEE Trans. Knowl. Data Eng. 35 (4) (2023) 4216–4235.
[10] P. Veličković, W. Fedus, W.L. Hamilton, et al., Deep graph infomax, in: 7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6–9, 2019, 2019, pp. 1–17.
[11] K. Hassani, A.H. Khasahmadi, Contrastive multi-view representation learning on graphs, in: Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13–18 July 2020, Virtual Event, 2020, pp. 4116–4126.

[12] Y. Zhu, Y. Xu, F. Yu, et al., Deep graph contrastive representation learning, in: ICML Workshop on Graph Representation Learning and Beyond, 2020, pp. 1–17.
[13] Y. You, T. Chen, Y. Sui, et al., Graph contrastive learning with augmentations, in: Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6–12, 2020, Virtual, 2020, pp. 5812–5823.
[14] Q. Zhu, B. Du, P. Yan, Self-supervised training of graph convolutional networks, arXiv:2006.02380(2020).
[15] Y. Jiao, Y. Xiong, J. Zhang, et al., Sub-graph contrast for scalable self-supervised graph representation learning, in: 20th IEEE International Conference on Data Mining, ICDM 2020, Sorrento, Italy, November 17–20, 2020, 2020, pp. 222–231.
[16] M. Jin, Y. Zheng, Y.-F. Li, et al., Multi-scale contrastive siamese networks for self-supervised graph representation learning, in: Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI 2021, Virtual Event / Montreal, Canada, 19–27 August 2021, 2021, pp. 1477–1483.
[17] M.I. Belghazi, A. Baratin, S. Rajeshwar, et al., Mutual information neural estimation, in: Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10–15, 2018, 2018, pp. 530–539.
[18] S. Nowozin, B. Cseke, R. Tomioka, f-GAN: Training generative neural samplers using variational divergence minimization, in: Advances in Neural Information Processing Systems 29: Annual Conference on Neural Information Processing Systems 2016, December 5–10, 2016, Barcelona, Spain, 2016, pp. 271–279.
[19] M. Gutmann, A. Hyvärinen, Noise-contrastive estimation: A new estimation principle for unnormalized statistical models, in: Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, AISTATS 2010, Chia Laguna Resort, Sardinia, Italy, May 13–15, 2010, 2010, pp. 297–304.
[20] F. Schroff, D. Kalenichenko, J. Philbin, Facenet: A unified embedding for face recognition and clustering, in: IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2015, Boston, MA, USA, June 7–12, 2015, 2015, pp. 815–823.
[21] E.D. Cubuk, B. Zoph, D. Mane, et al., Autoaugment: Learning augmentation strategies from data, in: IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, Long Beach, CA, USA, June 16–20, 2019, 2019, pp. 113–123.
[22] D. Kim, J. Baek, S.J. Hwang, Graph self-supervised learning with accurate discrepancy learning, in: Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28–December 9, 2022, 2022, pp. 1–19.
[23] W.L. Hamilton, R. Ying, J. Leskovec, Representation learning on graphs: Methods and applications, IEEE Data Eng. Bull. 40 (2017) 52–74.
[24] K. Sun, L. Wang, B. Xu, et al., Network representation learning: From traditional feature learning to deep learning, IEEE Access 8 (2020) 205600–205617.
[25] B. Perozzi, R. Al-Rfou, S. Skiena, Deepwalk: Online learning of social representations, in: The 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '14, New York, NY, USA - August 24–27, 2014, 2014, pp. 701–710.
[26] J. Tang, M. Qu, M. Wang, et al., Line: Large-scale information network embedding, in: Proceedings of the 24th International Conference on World Wide Web, WWW 2015, Florence, Italy, May 18–22, 2015, 2015, pp. 1067–1077.
[27] A. Grover, J. Leskovec, node2vec: Scalable feature learning for networks, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13–17, 2016, 2016, pp. 855–864.
[28] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: 5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Conference Track Proceedings, 2017, pp. 1–14.
[29] W.L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4–9, 2017, Long Beach, CA, USA, 2017, pp. 1024–1034.
[30] P. Veličković, G. Cucurull, A. Casanova, et al., in: 6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30–May 3, 2018, Conference Track Proceedings, 2018, pp. 1–12.
[31] K. Xu, W. Hu, J. Leskovec, et al., How powerful are graph neural networks? in: 7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6–9, 2019, 2019, pp. 1–17.
[32] C. Morris, M. Ritzert, M. Fey, et al., Weisfeiler and Leman go neural: Higher-order graph neural networks, in: The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, Honolulu, Hawaii, USA, January 27–February 1, 2019, 2019, pp. 4602–4609.
[33] X. Ma, L. Gao, X. Yong, Eigenspaces of networks reveal the overlapping and hierarchical community structure more precisely, J. Stat. Mech. 2010 (08) (2010) P08012.
[34] B. Bevilacqua, F. Frasca, D. Lim, et al., Equivariant subgraph aggregation networks, in: The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25–29, 2022, 2022, pp. 1–46.
[35] P. Bielak, T. Kajdanowicz, N.V. Chawla, Graph barlow twins: A self-supervised representation learning framework for graphs, Knowledge-Based Syst. 256 (2022) 109631.
[36] S. Suresh, P. Li, C. Hao, et al., Adversarial graph augmentation to improve graph contrastive learning, in: Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6–14, 2021, virtual, 2021, pp. 15920–15933.
[37] S. Feng, B. Jing, Y. Zhu, et al., Adversarial graph contrastive learning with information regularization, in: WWW '22: The ACM Web Conference 2022, Virtual Event, Lyon, France, April 25–29, 2022, 2022, pp. 1362–1371.
[38] Y. Zhu, Y. Xu, F. Yu, et al., Graph contrastive learning with adaptive augmentation, in: WWW '21: The Web Conference 2021, Virtual Event / Ljubljana, Slovenia, April 19–23, 2021, 2021, pp. 2069–2080.
[39] T. Chen, S. Kornblith, M. Norouzi, et al., A simple framework for contrastive learning of visual representations, in: Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13–18 July 2020, Virtual Event, 2020, pp. 1597–1607.
[40] X. Shen, D. Sun, S. Pan, et al., Neighbor contrastive learning on learnable graph augmentation, in: Thirty-Seventh AAAI Conference on Artificial Intelligence, Washington, DC, USA, February 7–14, 2023, 2023, pp. 9782–9791.
[41] J. Chen, G. Kou, Attribute and structure preserving graph contrastive learning, in: Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023, Washington, DC, USA, February 7–14, 2023, 2023, pp. 7024–7032.
[42] H. Chen, Z. Zhao, Y. Li, et al., Csgcl: Community-strength-enhanced graph contrastive learning, in: Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI 2023, 19th–25th August 2023, Macao, SAR, China, 2023, pp. 2059–2067.
[43] Y. Yang, X. Ma, Graph contrastive learning for clustering of multi-layer networks, IEEE Trans. Big Data 10 (2023) 1–13.
[44] H. Wang, W. Zhang, X. Ma, Contrastive and adversarial regularized multi-level representation learning for incomplete multi-view clustering, Neural Netw. 172 (2024) 106102.
[45] H. Wang, Q. Wang, Q. Miao, et al., Joint learning of data recovering and graph contrastive denoising for incomplete multi-view clustering, Inf. Fusion 104 (2024) 102155.
[46] Y. Ai, X. Xie, X. Ma, Graph contrastive learning for tracking dynamic communities in temporal networks, IEEE Trans. Emerg. Top. Comput. Intell. Early Access (2024) 1–14.
[47] X. Cai, C. Huang, L. Xia, et al., Lightgcl: Simple yet effective graph contrastive learning for recommendation, in: The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1–5, 2023, 2023, pp. 1–15.
[48] Y. Yin, Q. Wang, S. Huang, et al., Autogcl: Automated graph contrastive learning via learnable view generators, in: Thirty-Sixth AAAI Conference on Artificial Intelligence, AAAI 2022, February 22–March 1, 2022, 2022, pp. 8892–8900.
[49] A. Rajwade, A. Rangarajan, A. Banerjee, Image denoising using the higher order singular value decomposition, IEEE Trans. Pattern Anal. Mach. Intell. 35 (4) (2012) 849–862.
[50] A. Rangarajan, Learning matrix space image representations, in: Energy Minimization Methods in Computer Vision and Pattern Recognition, Third International Workshop, EMMCVPR 2001, Sophia Antipolis, France, September 3–5, 2001, Proceedings, 2001, pp. 153–168.
[51] E. Jang, S. Gu, B. Poole, Categorical reparameterization with gumbel-softmax, in: 5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Conference Track Proceedings, 2017, pp. 1–12.
[52] R.D. Hjelm, A. Fedorov, et al., Learning deep representations by mutual information estimation and maximization, in: 7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6–9, 2019, 2019, pp. 1–24.
[53] J. McAuley, C. Targett, Q. Shi, et al., Image-based recommendations on styles and substitutes, in: Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval, Santiago, Chile, August 9–13, 2015, 2015, pp. 43–52.
[54] O. Shchur, M. Mumme, A. Bojchevski, et al., Pitfalls of graph neural network evaluation, Relational Representation Learning Workshop, NeurIPS 2018, 2018.
[55] A. Paszke, S. Gross, F. Massa, et al., Pytorch: An imperative style, high-performance deep learning library, in: Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8–14, 2019, Vancouver, BC, Canada, 2019, pp. 8024–8035.
[56] M. Fey, J.E. Lenssen, Fast graph representation learning with PyTorch geometric, in: ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019, pp. 1–9.
[57] L. Van der Maaten, G. Hinton, Visualizing data using t-SNE, J. Mach. Learn. Res. 9 (86) (2008) 2579–2605.

Author profile

Pengfei Jiao received the Ph.D. degree in computer science from Tianjin University, Tianjin, China, in 2018. From 2018 to 2021, he was a lecturer with the Center of Biosafety Research and Strategy of Tianjin University. He is currently a professor with the School of Cyberspace, Hangzhou Dianzi University, Hangzhou, China. His current research interests include complex network analysis and its applications.

Qing Bao (BRID: 06366.00.23587) received the B.Sc. degree from the Department of Computer Science and Technology, East China Normal University, Shanghai, China, in 2011, and the Ph.D. degree in computer science from Hong Kong Baptist University, Hong Kong, in 2016. She is currently an associate professor with the School of Cyberspace, Hangzhou Dianzi University, China. Before that, she was a Post-doctoral Research Fellow with Hong Kong Baptist University. Her research interests include graph data mining, social network analysis, and health informatics. She was a recipient of the Best Student Paper Award in the 2013 IEEE/WIC/ACM International Conference on Web Intelligence. She is a reviewer of various journals and a program committee member of several conferences.